Lasy (B)

Limit pamięci: 256 MB Limit czasu: 2.00 s

Rebelianci budują swoją bazę na planecie Yavin IV. Jest to planeta leśna, gęsto obrośnięta lasem. Żeby umożliwić lądowanie statków, transport materiałów między placówkami, a także unikać niebezpiecznych stworów, dowództwo postanowiło wyciąć pewną część lasu.

Nie jest jednak lekko. Planeta Yavin znana jest ze swojej gęstej roślinności, zatem przy wycięciu zbyt dużej ilości lasów Imperium szybko zorientuje się, że coś się nie zgadza i wyśle tam zwiadowców, co skutkować będzie najprawdopodobniej zdradzeniem lokalizacji bazy. Dla uproszczenia fragment planety osiedlony przez rebeliantów można zaprezentować jako układ współrzędnych. Generalicja wie, że jeśli zdjęcia skanujące wykażą, że pewien obszar pokrywa mniej niż P procent lasu, wrogowie staną się podejrzliwi. Wiadomo na szczęście jaki obszar jest skanowany.

Niestety kolejnym utrudnieniem jest fakt, że niekiedy zdjęcia są przeglądane ręcznie, a nie analizowane komputerowo, więc nienaturalne ułożenie lasów może wzbudzić podejrzenia. Główny florolog orzekł, że lasy najgęściej rosną w pobliżu rzek. Ze względu na małą dokładność skanów, przyjęto że rzeki są równoległe do osi układu współrzędnych. Decyzją florologa wycięte zostaną wszystkie drzewa w odległości większej niż R od wszystkich rzek. Odległość punktu $(X,\ Y)$ od rzeki definiujemy jako $\min_{(X_i,\ Y_i)}(\max(|X-X_i|,\ |Y-Y_i|))$, gdzie punkty (X_i, Y_i) należą do rzeki. Przykładowo, wszystkie punkty odległe o co najwyżej 1 od rzeki o końcach $(0,\ 0)$, $(0,\ 2)$ utworzą prostokąt o rogach $(-1,\ -1)$, $(1,\ 3)$.

Twoim zadaniem, jako głównego analityka, jest wybór najmniejszego naturalnego R takiego, by zwiadowcy Imperium nie nabrali podejrzeń.

Wejście

Pierwszy wiersz wejścia zawiera jedną liczbę N oznaczającą liczbę rzek na Yavin IV. Kolejne N wierszy zawiera po cztery liczby: X_{i1}, Y_{i1}, X_{i2}, Y_{i2}, takie, że $(X_{i1},\ Y_{i1})$ i $(X_{i2},\ Y_{i2})$ są końcami pewnej rzeki. Jako że rzeki przyjmujemy za równoległe do osi układu współrzędnych, to $X_{i1} = X_{i2}$ lub $Y_{i1} = Y_{i2}$. W następnym wierszu znajduje się liczba P mówiąca jaki procent powierzchni musi pozostać zarośnięty lasami. W ostatnim wierszu znajdują się cztery liczby: A_1, B_1, A_2, B_2, takie, że $(A_1,\ B_1)$ jest lewym dolnym rogiem skanowanego przez Imperium terenu, a $(A_2,\ B_2)$ prawym górnym.

Wyjście

W jedynym wierszu wyjścia powinna znaleźć się jedna liczba, wskazująca najmniejszy promień R spełniające warunki podane w treści.

Ograniczenia

$1 \leq N \leq 10\,000$, $0 \leq X_{i1}, Y_{i1}, X_{i2}, Y_{i2} \leq 100\,000$, $1 \leq P \leq 100$, $0 \leq A_1 < A_2 \leq 100\,000$, $0 \leq B_1 < B_2 \leq 100\,000$.

Podzadania

Podzadanie	Warunki	Punkty
1	$N = 1$, $A_2, B_2 \leq 1000$	10
2	$N = 1$	10
3	$N = 2$	10
4	$A_2, B_2 \leq 1000$	30
5	brak dodatkowych ograniczeń	40

Przykład

Wejście	Wyjście

```
3
1 1 4 1
2 2 2 8
3 2 7 2
50
1 1 15 15
```

```
5
```

Wejście

```
1
0 0 0 4
50
0 0 4 4
```

Wyjście

```
2
```

Para (BB)

Limit pamięci: 256 MB Limit czasu: 0.50 s

To zadanie jest interaktywne. Po wypisaniu każdego wiersza należy opróżnić bufor. W C++ możesz użyć `cout << flush`, w Pythonie – `sys.stdout.flush()`. Należy ściśle trzymać się protokołu opisanego w sekcji *Interakcja* – niewczytanie wiadomości wysłanej przez interaktor skutkuje werdyktem *wrong answer* lub innym.

Dana jest liczba N oraz pewna ukryta para liczb całkowitych $1 \leq A < B \leq N$.
Należy odnaleźć parę, używając zapytań postaci:

- ? k $s_1 \ldots s_k$ oznacza zapytanie o to, czy w zbiorze k liczb $\{s_1, \ldots, s_k\}$ znajdują się obie spośród A, B. W odpowiedzi dostajemy napis `TAK` lub `NIE`.

Liczba punktów zależy od sumy wartości k po wszystkich zapytaniach. Żeby dostać dodatnią liczbę punktów, nie może ona przekroczyć $10\,000$.

Interakcja

Na początku należy wczytać liczby całkowite N, T. Wartość $0 \leq T \leq 5$ oznacza numer podzadania[1], przy czym $T = 0$ oznacza test przykładowy.

Następnie należy wykonać ciąg zapytań, każde poprzez wypisanie odpowiedniego wiersza, a następnie **opróżnienie bufora**. Każde zapytanie zaczyna się od znaku `?`. Po nim powinna wystąpić liczba k oznaczająca liczbę elementów podzbioru z zapytania oraz ciąg k *parami różnych* elementów. W odpowiedzi należy wczytać napis `TAK` lub `NIE`.

Po wykonaniu wszystkich zapytań należy wypisać `!` A B, gdzie $\{A, B\}$ to znaleziona para (kolejność bez znaczenia), opróżnić bufor i zakończyć działanie programu. Nie należy już nic więcej wypisywać.

Ograniczenia

$2 \leq N \leq 100$, $1 \leq A < B \leq N$.

Podzadania

Interaktor jest adaptacyjny tylko w ostatnim podzadaniu. Oznacza to, że we wcześniejszych podzadaniach para (A, B) jest ustalona z góry, natomiast w ostatnim podzadaniu może się ona zmieniać w zależności od zadanych zapytań (ale nowa para jest zgodna z odpowiedziami na wszystkie poprzednie zapytania).

Niech Q oznacza największą sumę wartości k po zapytaniach zadanych przez Twój program w danym podzadaniu. Wówczas procent punktów uzyskanych za to podzadanie jest obliczany na podstawie poniższej tabeli:

Wartość Q	Procent punktów
$Q > 10\,000$	0
$1000 \leq Q \leq 10\,000$	$15 + 35 \cdot \frac{10\,000 - Q}{10\,000 - 1000}$
$400 \leq Q \leq 1000$	$50 + 42 \cdot \frac{1000 - Q}{1000 - 400}$
$305 \leq Q \leq 400$	$92 + 8 \cdot \frac{400 - Q}{400 - 305}$
$Q \leq 305$	100

Poniższa tabelka zawiera dodatkowe warunki spełniane przez podzadania.

T	**Adaptacja**	**Dodatkowe warunki**	**Punkty**
1	NIE	$A = 1$	10
2	NIE	$A \leq 10 < B$	20

[1] Można dzięki temu napisać różne algorytmy do różnych podzadań i wyifować, którego z nich użyć.

T	Adaptacja	Dodatkowe warunki	Punkty
3	NIE	$A \leq \frac{N}{2} < B$	20
4	NIE	brak	30
5	TAK	brak	20

Przykładowa interakcja

Wejście	Wyjście	Wyjaśnienie
`5 0`		Mamy $N = 5$ elementów i jest to test przykładowy
	`? 4 1 2 3 4`	
`TAK`		$1, 4 \in \{1, 2, 3, 4\}$
	`? 4 1 2 3 5`	
`NIE`		$4 \notin \{1, 2, 3, 5\}$
	`? 2 4 5`	
`NIE`		$1 \notin \{4, 5\}$
	`! 1 4`	Para $(1, 4)$ została znaleziona

Odpowiada ona **pierwszemu testowi przykładowemu**. Suma rozmiarów zbiorów z zapytań to $4 + 4 + 2 = 10$.

Narzędzia do testowania lokalnego i testy przykładowe

W zadaniu są 3 testy przykładowe. Zostanie również udostępniony program **interaktor.cpp** oraz przykładowe błędne rozwiązanie o poprawnym schemacie komunikacji **abc.cpp**. Żeby uruchomić rozwiązanie na teście, należy skompilować rozwiązanie i interaktorkę, a następnie użyć komendy:

`[plik wykonywalny z interaktor.cpp] [test] [plik wykonywalny z rozwiązaniem]`

Na przykład: `./interaktor 0a.in ./abc`

Wspinaczka (BBB)

Limit pamięci: 256 MB Limit czasu: 5.00 s

Jasio wybiera się w góry. Możemy je przedstawić jako graf nieskierowany, w którym wierzchołki oznaczają szczyty górskie, a krawędzie – łączące je szlaki. Szczyt o numerze i jest położony na wysokości i.

Jasio najbardziej lubi takie trasy, w których najpierw przez jakiś czas odwiedzamy coraz wyższe szczyty, a następnie coraz niższe, nie odwiedzając przy tym żadnego szczytu dwa razy. Formalnie, *dobra* trasa to taka ścieżka prosta $v_1, v_2, \ldots, v_k$ $(1 \leq k)$, że istnieje indeks $s\,(1 \leq s \leq k)$, taki że wysokości szczytów odpowiadających kolejnym wierzchołkom tworzą ciąg bitoniczny $v_1 < \cdots < v_{s-1} < v_s > v_{s+1} > \cdots > v_k$.

Pomóż Jasiowi zaplanować wycieczkę obliczając liczbę szlaków na najdłuższej dobrej trasie.

Wejście

W pierwszym wierszu wejścia znajdują się dwie liczby całkowite N i M, oznaczające liczbę szczytów górskich oraz liczbę szlaków. W kolejnych M wierszach wejścia znajduje się opis szlaków. Każdy z nich zawiera dwie liczby, oznaczające szlak między szczytami o takich numerach.

Możesz założyć, że z każdego szczytu da się dojść do wszystkich pozostałych.

Wyjście

Twój program powinien wypisać jedną liczbę całkowitą, oznaczającą liczbę szlaków na najdłuższej dobrej trasie.

Ograniczenia

$1 \leq N \leq 5\,000$, $1 \leq M \leq 20\,000$.

Podzadanie	Punkty	Warunki
1	10	$N \leq 9, M \leq 50$
2	10	$N \leq 15, M \leq 100$
3	10	$N \leq 400, M \leq 1\,600$
4	10	$N \leq 800, M \leq 3\,200$
5	10	$N \leq 1\,600, M \leq 6\,400$
6	50	brak dodatkowych ograniczeń

Przykład

Wejście

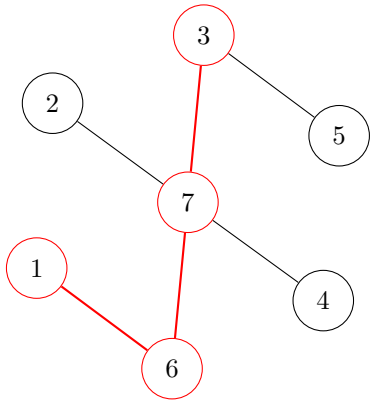
```
7 6
7 2
7 3
7 6
3 5
1 6
4 7
```

Wyjście

```
4
```

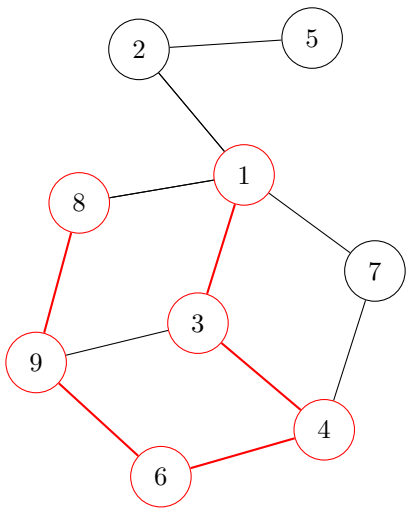
Wyjaśnienie

Na wejściu mamy podane drzewo o 7 wierzchołkach i 6 krawędziach. Przykładowo, Jasiu może obrać następujący szlak bitoniczny o długości 4: $3 < 7 > 6 > 1$.



Wejście

```
9 14
7 4
9 3
6 4
2 1
8 1
5 2
2 1
1 3
9 6
9 8
8 1
7 1
9 9
4 3
```

Wyjście

6